Fiche d'exercices — Programmation orientée objet en Python

Classes, attributs, méthodes · Encapsulation et héritage · BTS CIEL corrigé

Version enseignant — corrigés intégrés

> **Mémo** : **Classe** : modèle. **Objet** (instance) : réalisation concrète de ce modèle. **Attribut** : donnée portée par l'objet. **Méthode** : fonction définie dans la classe. `self` désigne l'objet sur lequel la méthode est appelée ; `__init__` est le **constructeur**. `return` *retourne* une valeur, `print` l'*affiche*.

Partie A — Classes, attributs et méthodes

Exercice 1 — La classe `Voiture`

```python
class Voiture:
    def __init__(self, marque, couleur):
        self.marque = marque
        self.couleur = couleur

    def afficher_details(self):
        return f"Voiture {self.marque} de couleur {self.couleur}"
```

Questions

1. Que fait la méthode `__init__` ?

   C'est le **constructeur**. Il est appelé automatiquement à la création de l'objet et initialise les attributs d'instance `self.marque` et `self.couleur` à partir des valeurs passées en paramètres.

2. Combien d'attributs possède la classe `Voiture` ?

   Deux attributs d'instance : `marque` et `couleur`.

3. Que retourne la méthode `afficher_details` ?

   Une chaîne de caractères (`str`) construite par une f-string, par exemple `"Voiture Toyota de couleur Bleu"`. Elle *retourne* la chaîne, elle ne l'affiche pas.

4. Créez une instance de `Voiture` de marque `"Toyota"` et de couleur `"Bleu"`, appelez `afficher_details` et affichez le résultat.

Corrigé

```python
ma_voiture = Voiture("Toyota", "Bleu")
print(ma_voiture.afficher_details())      # Voiture Toyota de couleur Bleu
```

`print` est nécessaire : sans lui, la chaîne est retournée puis perdue.

Exercice 2 — Compléter la classe `Chien`

```python
class Chien:
    def __init__(self, nom, age):
        self.nom = nom
        self.age = age

    # Méthode aboyer qui retourne "Wouf !"
```

Questions

1. Quels sont les attributs de la classe `Chien` ?

   `nom` et `age`, deux attributs d'instance.

2. Comment appelle-t-on la méthode `__init__` dans le contexte de la POO ?

   Le **constructeur** (ou initialiseur). C'est une *méthode spéciale* ; on ne l'appelle jamais directement, elle est déclenchée par `Chien("Rex", 3)`.

3. Ajoutez une méthode `aboyer` qui retourne `"Wouf !"`.
4. Créez un objet `Chien` nommé `"Rex"`, âgé de 3 ans, et faites-le aboyer.

Corrigé

```python
class Chien:
    def __init__(self, nom, age):
        self.nom = nom
        self.age = age

    def aboyer(self):
        return "Wouf !"

rex = Chien("Rex", 3)
print(rex.aboyer())        # Wouf !
```

Le paramètre `self` est obligatoire dans la définition, même si `aboyer` n'utilise aucun attribut : il reçoit l'objet sur lequel la méthode est appelée.

Exercice 3 — La classe `Livre`

```python
class Livre:
    def __init__(self, titre, auteur):
        self.titre = titre
        self.auteur = auteur

    def description(self):
        return f"{self.titre} par {self.auteur}"
```

Questions

1. Que fait la méthode `description` ?

   Elle retourne une chaîne qui associe le titre et l'auteur du livre, sous la forme `"titre par auteur"`.

2. Combien de paramètres la méthode `__init__` attend-elle, en incluant `self` ?

   Trois : `self`, `titre` et `auteur`. À l'appel, seuls deux arguments sont fournis : `self` est transmis automatiquement par Python.

3. Créez un objet `Livre` de titre `"Le Petit Prince"` et d'auteur `"Antoine de Saint-Exupéry"`, puis affichez sa description.

Corrigé

```python
livre = Livre("Le Petit Prince", "Antoine de Saint-Exupery")
print(livre.description())      # Le Petit Prince par Antoine de Saint-Exupery
```

Exercice 4 — La classe `Rectangle`

```python
class Rectangle:
    def __init__(self, longueur, largeur):
        self.longueur = longueur
        self.largeur = largeur

    # Méthode calculer_aire qui retourne l'aire
```

Questions

1. Quels sont les attributs de la classe `Rectangle` ?

   `longueur` et `largeur`.

2. Quelle formule mathématique devez-vous utiliser pour calculer l'aire ?

   aire = longueur × largeur.

3. Ajoutez une méthode `calculer_aire` qui retourne l'aire du rectangle.
4. Créez un rectangle de longueur 5 et de largeur 3, puis affichez son aire.

Corrigé

```python
class Rectangle:
    def __init__(self, longueur, largeur):
        self.longueur = longueur
        self.largeur = largeur

    def calculer_aire(self):
        return self.longueur * self.largeur

r = Rectangle(5, 3)
print(r.calculer_aire())        # 15
```

Une méthode peut aussi être ajoutée pour le périmètre :
`return 2 * (self.longueur + self.largeur)`.

Exercice 5 — Création complète d'une classe `Etudiant`

On souhaite modéliser un étudiant par son nom et sa note.

Questions

1. Créez une classe `Etudiant` avec un constructeur `__init__` prenant en paramètres `nom` et `note`, et deux attributs d'instance `self.nom` et `self.note`.
2. Ajoutez une méthode `a_reussi` qui retourne `True` si la note est supérieure ou égale à 10, et `False` sinon.
3. Testez votre classe en créant un étudiant avec une note de 8.
4. Ajoutez une méthode `augmenter_note` qui augmente la note de 2 points.
5. Créez un étudiant nommé `"Alice"` avec une note de 15 et affichez si elle a réussi, grâce à la méthode `a_reussi`.

Corrigé

```python
class Etudiant:
    def __init__(self, nom, note):
        self.nom = nom
        self.note = note
```

```python
    def a_reussi(self):
        return self.note >= 10

    def augmenter_note(self):
        self.note = self.note + 2

e = Etudiant("Bob", 8)          # 3. test avec une note de 8
print(e.a_reussi())             # False

alice = Etudiant("Alice", 15)   # 5. Alice
print(alice.a_reussi())         # True
```

Points d'attention. `return self.note >= 10` suffit : la comparaison produit déjà un booléen, un `if ... return True else return False` est inutile. `augmenter_note` *modifie* l'attribut et ne retourne rien : elle agit sur l'état de l'objet.

Partie B — Encapsulation et héritage

Mémo : **Encapsulation** : `self.__nom` (deux soulignés) déclenche le *name mangling* — Python renomme l'attribut en `_Classe__nom`, donc inaccessible de l'extérieur par son nom d'origine. On y accède par un **accesseur** (`get_nom`) et on le modifie par un **mutateur** (`set_nom`), qui peut valider la valeur. **Héritage** : `class Chat(Animal)` — la classe fille reçoit les attributs et méthodes de la mère, peut en ajouter ou **redéfinir** les siennes ; `super().__init__(...)` appelle le constructeur de la mère.

Exercice 6 — Encapsulation simple avec attribut privé

```python
class Personne:
    def __init__(self, nom, age):
        self.__nom = nom
        self.__age = age

    def get_nom(self):
        return self.__nom

    # Méthode get_age()
```

Questions

1. Pourquoi les attributs `__nom` et `__age` sont-ils privés ?

   Pour empêcher leur modification directe depuis l'extérieur. L'accès passe par des méthodes, ce qui permet de contrôler la lecture et l'écriture et de modifier l'implémentation interne sans casser le code qui utilise la classe.

2. Comment accède-t-on à l'attribut `__nom` ?

   Par l'accesseur : `p.get_nom()`. L'écriture `p.__nom` provoque une `AttributeError`.

3. Ajoutez une méthode `get_age` pour accéder à l'attribut `__age`.
4. Créez une instance de `Personne` de nom `"Luc"` et d'âge 25, puis affichez son nom.

Corrigé

```python
    def get_age(self):
        return self.__age

p = Personne("Luc", 25)
print(p.get_nom())        # Luc
print(p.get_age())        # 25
```

Exercice 7 — Ajouter un mutateur

```python
class CompteBancaire:
    def __init__(self, titulaire, solde):
        self.__titulaire = titulaire
        self.__solde = solde

    def get_solde(self):
        return self.__solde
```

Questions

1. Que signifie l'encapsulation dans cette classe ?

   Le titulaire et le solde sont privés : aucun code extérieur ne peut écrire `compte.__solde = 1000000`. Le solde ne peut être consulté que par `get_solde` et modifié que par les méthodes que la classe met à disposition.

2. Que fait la méthode `get_solde` ?

   Elle retourne la valeur de l'attribut privé `__solde` : c'est un accesseur en lecture seule.

3. Créez un compte bancaire pour `"Emma"` avec un solde de 500 et affichez le solde.
4. Ajoutez une méthode `set_solde` pour modifier le solde.

Corrigé

```
    def set_solde(self, nouveau_solde):
        self.__solde = nouveau_solde

compte = CompteBancaire("Emma", 500)
print(compte.get_solde())       # 500
compte.set_solde(750)
print(compte.get_solde())       # 750
```

Prolongement. Un mutateur brut n'apporte aucune sécurité. Une version plus réaliste refuse un solde négatif, ou expose plutôt des opérations métier : `deposer(montant)` et `retirer(montant)` avec vérification de la provision.

Exercice 8 — Encapsulation avec validation

```
class Etudiant:
    def __init__(self, nom, note):
        self.__nom = nom
        self.__note = note

    def get_note(self):
        return self.__note

    # Méthode set_note : notes entre 0 et 20
```

Questions

1. Pourquoi utiliser un mutateur pour la note ?

   Pour garantir un *invariant* : une note reste comprise entre 0 et 20. Le mutateur est le seul point d'entrée en écriture, il peut donc vérifier la valeur avant de l'enregistrer.

2. Que se passe-t-il si on essaie d'accéder directement à `__note` ?

   Python lève une `AttributeError` : à cause du *name mangling*, l'attribut s'appelle en réalité `_Etudiant__note` et le nom `__note` n'existe pas depuis l'extérieur.

3. Ajoutez une méthode `set_note` qui n'accepte que des notes entre 0 et 20.
4. Créez un étudiant nommé `"Paul"` avec une note de 15, puis essayez de changer la note à 25.

Corrigé

```
    def set_note(self, nouvelle_note):
        if 0 <= nouvelle_note <= 20:
            self.__note = nouvelle_note
        else:
            print("Note invalide : elle doit etre comprise entre 0 et 20.")

paul = Etudiant("Paul", 15)
paul.set_note(25)         # Note invalide : ...
print(paul.get_note())    # 15 : la note n'a pas change
```

Variante professionnelle. Au lieu d'un `print`, lever une exception rend l'erreur impossible à ignorer : `raise ValueError("note hors intervalle")`.

Exercice 9 — Héritage simple

```
class Animal:
    def __init__(self, nom):
        self.__nom = nom

    def get_nom(self):
        return self.__nom

    def faire_son(self):
        return "Son generique"

class Chat(Animal):
    # Méthode faire_son pour retourner "Miaou"
```

Questions

1. Quelle classe est la classe parent ?

   `Animal`. `Chat` est la classe fille (ou classe dérivée).

2. Que fait l'héritage dans ce cas ?

   `Chat` récupère sans les réécrire le constructeur `__init__`, l'attribut privé `__nom` et la méthode `get_nom` d'`Animal`. Elle peut en plus redéfinir `faire_son` pour lui donner son comportement propre.

3. Redéfinissez la méthode `faire_son` pour retourner `"Miaou"`.
4. Créez un `Chat` nommé `"Minou"` et affichez son son.

Corrigé

```
class Chat(Animal):
    def faire_son(self):
```

```
        return "Miaou"

minou = Chat("Minou")
print(minou.get_nom())     # Minou  (methode heritee)
print(minou.faire_son())   # Miaou  (methode redefinie)
```

`Chat` ne définit pas de `__init__` : celui d'`Animal` est utilisé tel quel. Appeler `faire_son()` sur un `Chat` exécute la version de `Chat` : c'est le **polymorphisme**.

Exercice 10 — Héritage avec nouvel attribut

```
class Vehicule:
    def __init__(self, marque):
        self.__marque = marque

    def get_marque(self):
        return self.__marque

class Voiture(Vehicule):
    def __init__(self, marque, couleur):
        super().__init__(marque)
        self.__couleur = couleur

    # À compléter : méthode get_couleur
```

Questions

1. Que fait `super().__init__(marque)` ?

   Il appelle le constructeur de la classe mère `Vehicule` pour initialiser la partie héritée de l'objet, ici l'attribut privé `__marque`. Sans cet appel, `get_marque` échouerait car l'attribut n'aurait jamais été créé.

2. Combien d'attributs privés possède la classe `Voiture` ?

   Deux au total dans l'objet : `_Vehicule__marque`, hérité et initialisé par `super()`, et `_Voiture__couleur`, propre à `Voiture`. La classe `Voiture` n'en déclare qu'un seul elle-même.

3. À compléter : ajoutez une méthode `get_couleur`.

Corrigé

```
    def get_couleur(self):
        return self.__couleur

v = Voiture("Renault", "Bleu")
print(v.get_marque())      # Renault
print(v.get_couleur())     # Bleu
```

Piège. `__marque` est privé *à `Vehicule`* : à cause du *name mangling*, une méthode de `Voiture` ne peut pas écrire `self.__marque`. Elle doit passer par `self.get_marque()`. C'est exactement le rôle de l'accesseur hérité.

Exercice 11 — Encapsulation et mutateur avec validation

```
class Livre:
    def __init__(self, titre, prix):
        self.__titre = titre
        self.__prix = prix

    def get_prix(self):
        return self.__prix

    # À compléter : set_prix refuse les prix négatifs
```

Questions

1. Pourquoi le prix est-il un attribut privé ?

   Parce qu'un prix doit respecter une règle métier : il ne peut pas être négatif. Le rendre privé interdit l'écriture directe et oblige à passer par une méthode qui applique cette règle.

2. Que fait un mutateur avec validation ?

   Il vérifie la valeur reçue avant de l'affecter. Si elle est valide, il met l'attribut à jour ; sinon il refuse la modification et l'objet conserve son état précédent, qui reste cohérent.

3. Créez un `Livre` intitulé `"Python 101"` avec un prix de 30, puis essayez de définir un prix de -10.

Corrigé

```
    def set_prix(self, nouveau_prix):
        if nouveau_prix >= 0:
            self.__prix = nouveau_prix
        else:
            print("Prix invalide : un prix ne peut pas etre negatif.")

livre = Livre("Python 101", 30)
livre.set_prix(-10)        # Prix invalide : ...
print(livre.get_prix())    # 30 : le prix est inchange
```

Remarque. Le constructeur devrait appliquer la même validation, par exemple en appelant `self.set_prix(prix)` au lieu d'affecter directement : sinon `Livre("X", -10)` crée un objet invalide dès le départ.

Synthèse

Notion	Ce qu'il faut retenir
Classe	Modèle décrivant des données et des comportements : `class Voiture:`
Objet / instance	Réalisation concrète de la classe : `v = Voiture("Toyota", "Bleu")`
Constructeur	`__init__`, appelé automatiquement, initialise les attributs
`self`	Référence à l'objet courant ; premier paramètre de toute méthode d'instance
Attribut d'instance	Donnée propre à l'objet : `self.marque`
Méthode	Fonction définie dans la classe, appelée par `objet.methode()`
Attribut privé	`self.__prix` : inaccessible depuis l'extérieur (*name mangling*)
Accesseur (*getter*)	`get_prix()` : lit un attribut privé
Mutateur (*setter*)	`set_prix(v)` : écrit un attribut privé après validation
Héritage	`class Voiture(Vehicule)` : la fille reçoit les membres de la mère
`super()`	Accède à la classe mère, notamment `super().__init__(...)`
Redéfinition	La fille réécrit une méthode de la mère (`faire_son`)

Erreurs les plus fréquentes. Oublier `self` dans la définition d'une méthode ; confondre `return` (retourne une valeur) et `print` (affiche) ; écrire `objet.__attribut` depuis l'extérieur ; oublier `super().__init__(...)` dans le constructeur d'une classe fille ; appeler une méthode sans les parenthèses (`v.afficher_details` retourne la méthode, pas son résultat).